Auditoría técnica integral, seguridad y optimización — FRUTASO / FRUITES

Fecha: 6 de noviembre de 2026 · **Alcance:** código fuente del ZIP `frutaso---snacks-de-frutas-tropicales-deshidratadas` (67 archivos, sin `node_modules`) · **Fase 1:** solo diagnóstico. No se modificó ni eliminó ningún archivo del proyecto original.

0. Qué se probó y qué no (transparencia metodológica)

Ejecutado en sandbox (resultado real):

- `npm install` → **FALLA** con `ERESOLVE` (conflicto `esbuild ^0.25` vs peer de `vite 8.3.6`, que pide `^0.27 || ^0.28`).
- `npm install --legacy-peer-deps` → OK (194 paquetes). Solo se usó para poder auditar y compilar.
- `npm audit` → 0 vulnerabilidades conocidas en dependencias (al 6-nov-2026).
- `npm run lint` (`tsc --noEmit`) → exit 0, pero **no valida nada útil**: `tsconfig.json` tiene `allowJs` sin `checkJs`, y todo el código es `.jsx/.js`.
- `npm run build` → OK. Bundle JS único de **832,67 KB (229,71 KB gzip)**, CSS 8,35 KB, `dist/` = 6,9 MB.
- Pruebas automatizadas: **no existen** (ni unitarias, ni e2e, ni ESLint).

Revisado solo por lectura de código (no ejecutado): todo el comportamiento en navegador, el flujo contra Supabase real, las políticas RLS en una base viva, la Edge Function con Gemini, el diseño renderizado, el responsive en dispositivos.

No verificable sin ejecutar el sitio: LCP, INP, CLS, contraste real de colores, comportamiento táctil, tiempos de respuesta. Abajo se indican solo riesgos probables derivados del código.

Secretos: no se encontraron claves reales en el código. `.env.example` solo tiene valores vacíos y `.gitignore` excluye `.env*`. Sí hay una **credencial de demostración incrustada y mostrada en pantalla** (hallazgo C-01); no se reproduce aquí.

1. Diagnóstico ejecutivo

Estado general: es una SPA React 19 + Vite 8 + Tailwind 4 con HashRouter, 13 páginas, panel de administración y backend opcional en Supabase (5 tablas con RLS, 1 Edge Function que llama a Gemini). Compila bien y el código es legible y comentado. La marca, los textos y la prudencia comercial (no inventar precios, no hacer claims de salud) están cuidados.

Nivel de madurez técnica: prototipo avanzado / demo funcional. La capa de interfaz está bastante completa; la capa de datos y seguridad **no está lista**: está diseñada para "nunca romperse" (siempre cae a `localStorage`), y ese mismo diseño esconde fallos reales.

Fortalezas

- Estructura clara (`pages / components / services / data`), componentes con responsabilidad razonable.
- Sin `dangerouslySetInnerHTML`, `eval` ni `innerHTML` en React; React escapa por defecto.
- Todos los `target="_blank"` públicos llevan `rel="noopener noreferrer"` (3 excepciones, en el admin).
- RLS activado en las 5 tablas; la clave de Gemini va en la Edge Function y no en el navegador.
- Dependencias sin vulnerabilidades conocidas; sin `console.log` residual.
- Política de contenido prudente (precios y gramajes "por definir", sin promesas médicas en el asistente).

Debilidades principales

1. El formulario de contacto **no entrega mensajes al negocio** en ninguno de los dos modos (C-02).
2. El control de acceso del panel admin es solo una bandera en `localStorage` (C-01).
3. El modelo de datos del frontend (camelCase) no coincide con el esquema SQL (snake_case): con Supabase conectado, hay pantallas que se rompen (C-04).
4. El despliegue automático a GitHub Pages falla hoy (C-05).
5. Datos ficticios (tiendas "aliadas", mensajes) presentados como reales (A-02).
6. SEO casi inexistente por HashRouter y falta de metadatos.

Preparación para producción: No apto. Con las correcciones de Prioridad 1 y 2 (estimación: 1–2 semanas de trabajo enfocado) puede pasar a "apto para lanzamiento con observaciones" como landing de validación con contacto por WhatsApp.

2. Evaluación por categorías (0–100)

☐ Categoría	Nota	Evidencia principal	
Arquitectura y código	58	Buena separación de carpetas y comentarios; pero capa de servicios con fallback silencioso, 56 KB en un solo `AdminDashboard.jsx`, 5 dependencias sin uso, `package.json` con nombre `react-example`, sin tests ni ESLint, `lint` vacío.	
Funcionalidad	45	Navegación, catálogo (filtro por fruta), recetas, mapa Leaflet, chat demo y WhatsApp funcionan por código. Formulario, admin, escritura en BD, asistente con Gemini y despliegue tienen fallos o no están verificados.	
Seguridad	30	Admin sin guard real, credencial demo pública, Edge Function sin rate-limit real y con inyección de prompt, XSS potencial en popups, RLS con detalles débiles. Sin cabeceras de seguridad.	
Diseño y UX/UI	70 (estimada)	Sistema de diseño con variables CSS, paleta coherente, estados de carga y de éxito. **No se renderizó**; nota basada en código/CSS. Textos `[POR DEFINIR]` pueden generar desconfianza.	
Rendimiento	45	Bundle único de 833 KB; 8 JPG de 650–955 KB (≈6,5 MB) sin `loading="lazy"` ni `width/height`; sin code-splitting; CSS de Leaflet desde unpkg.	
SEO	25	HashRouter (`#/catalogo`), un solo `<title>`/description para todo el sitio, sin `og:image`, canonical, sitemap, robots, favicon ni JSON-LD.	
Responsive design	65 (estimada)	Uso extendido de `grid auto-fit/minmax` y tokens fluidos. No probado en dispositivos; estilos casi todos inline (difícil de mantener).	

Categoría	Nota	Evidencia principal	
Escalabilidad	40	Catálogo/recetas/tiendas editables desde BD es buena idea, pero el desajuste de modelo, la ausencia de paginación, tests y tipos, y los estilos inline frenan el crecimiento.	

3. Registro de hallazgos

Leyenda: **[C]** = confirmado por lectura de código o por ejecución · **[P]** = riesgo potencial / requiere prueba en entorno real.

Críticos

ID	Hallazgo	Ubicación	Gravedad	Consecuencia	Solución
C-01 [C]	**El acceso al panel admin es una bandera en `localStorage`.** El dashboard solo comprueba que exista `fruites_admin_session`; no consulta `supabase.auth.getSession()` ni `admin_users`. Además hay una credencial demo hardcodeada y mostrada en pantalla, y un botón de “entrar directamente”.	`AdminDashboard.jsx:65-76`, `AdminLogin.jsx:30-35, 61-66, 116-121, 51`	CRÍTICO	Cualquiera abre el panel con una línea en consola. Si el sitio sale sin variables de Supabase, queda en modo demo con credencial pública. Con Supabase real, RLS protege los datos [P], pero el panel muestra datos falsos y engaña.	Proteger la ruta con `getSession()` + verificación en `admin_users`; escuchar `onAuthStateChange`; eliminar la credencial y el botón demo del build de producción (`import.meta.env.DEV`); no guardar `role` en cliente.
C-02 [C]	**El formulario de contacto no entrega mensajes.** En modo demo guarda en el `localStorage` del propio visitante y responde “éxito”. Con Supabase, hace `insert().select().single()`, pero la tabla no tiene política `SELECT` para anónimos: el `SELECT` posterior falla [P, comportamiento típico de PostgREST], el `catch` lo traga y vuelve a guardar en `localStorage` devolviendo `success: true`.	`messagesService.js:61-86`; `schema.sql:129-134`	CRÍTICO	Pérdida silenciosa de leads; el usuario cree que escribió y el negocio no recibe nada.	Insertar sin `.select()` (`{ returning: 'minimal' }`); en producción, no hacer fallback: mostrar error y ofrecer WhatsApp; probar contra un proyecto Supabase real.
C-03 [C]	**Todas las escrituras caen a `localStorage` si Supabase falla** (productos, tiendas, recetas, mensajes, borrados). `deleteStore` / `deleteRecipe` borran localmente aunque el borrado remoto falle.	`productsService.js:51-90`, `storesService.js:40-96`, `recipesService.js:48-109`	CRÍTICO	El administrador cree haber publicado o eliminado algo que nunca llegó a la base; datos divergentes entre navegadores.	Separar “modo demo” (decidido por configuración) de “error de red”; en errores reales lanzar excepción y mostrar toast; quitar el fallback fuera de demo.

ID	Hallazgo	Ubicación	Gravedad	Consecuencia	Solución
C-04 [C]	**Modelo de datos incompatible con la base.** El frontend usa `descripcionCorta`, `colorPrincipal`, `perfilSensorial`, `momentosDeConsumo`, `frutasUsadas`, `tiempoPrep`, `saboresDisponibles`, `esClienteB2B`; el SQL define `descripcion_corta`, `color_principal`, `perfil_sensorial`, etc. Con Supabase conectado: `product.perfilSensorial.sabor` lanza `TypeError` (pantalla en blanco, no hay ErrorBoundary); los `update()` envían columnas inexistentes y fallan (luego C-03 lo oculta).	`ProductDetail.jsx:208-216`, `aiService.js:95`, `productsService.js:51-77`, `schema.sql:17-100`	CRÍTICO	Detalle de producto roto, edición imposible, asistente con descripciones `undefined`.	Crear una capa de mapeo (`fromDb/toDb`) en los servicios o unificar nombres; añadir `ErrorBoundary`; usar optional chaining.
C-05 [C]	**El despliegue está roto hoy.** `npm install` falla por `ERESOLVE` (`esbuild ^0.25` vs vite 8). No hay `package-lock.json` (solo `bun.lock`) mientras el workflow usa `cache: 'npm'` (esa acción exige un lockfile de npm/yarn; no se ejecutó el workflow).	`package.json` devDependencies; `.github/workflows/deploy.yml:28-35`	CRÍTICO	No hay publicación automática; builds no reproducibles.	Eliminar `esbuild` de devDependencies (lo trae vite), generar y versionar `package-lock.json`, usar `npm ci`; o migrar el workflow a Bun.

Altos

ID	Hallazgo	Ubicación	Gravedad	Consecuencia	Solución
A-01 [C]	**Rutas absolutas de imágenes (`/images/...`) con `base: './'`.** Funcionan en un dominio raíz, pero se rompen si se publica en un subdirectorio, que es justo el caso que documenta el propio `vite.config.ts` (`usuario.github.io/frutaso/`).	`products.js:43,75,107,139`, `recipes.js:22,55,86,119`, `Home.jsx:170`, `schema.sql:180,197`	ALTO	Todas las fotos de producto y recetas dan 404 en GitHub Pages de proyecto.	Importar imágenes desde `src/assets` (Vite las fingerprinta) o usar `import.meta.env.BASE_URL`; decidir si se usa dominio propio.
A-02 [C]	**Datos ficticios presentados como reales.** Tiendas "aliadas" inventadas (con teléfonos y horarios) se muestran en el mapa bajo el texto "cafés, gimnasios y tiendas saludables que ofrecen nuestra Colección Tropical", y el SQL las inserta en la base de producción. Mensajes semilla con nombres inventados. El propio sitio dice que el producto está en formulación.	`stores.js`, `schema.sql:205-242`, `StoreLocator.jsx:132`, `messagesService.js:6-40`	ALTO	Riesgo reputacional y de publicidad engañosa frente al consumidor (consultar con abogado de consumo, Ley 1480/2011).	Retirar el seed de producción; mostrar el mapa vacío con mensaje "próximamente" hasta tener aliados reales.
A-03 [C]	**Edge Function de IA sin defensa real.** El límite de 20 mensajes vive en una variable del cliente (se reinicia al recargar). CORS `*`. El mensaje del usuario y el catálogo (enviado por el cliente) se concatenan en el mismo bloque que las instrucciones del sistema: inyección de prompt. La clave de Gemini va en la URL (queda en logs). Responde 200 incluso ante errores. Usa `deno.land/std@0.168.0`.	`aiService.js:15-17, 92-98`; `ai-assistant/index.ts:15-19, 56-80, 27-36`	ALTO	Abuso de cuota/costo; respuestas manipulables.	Usar `systemInstruction` separado; ignorar el catálogo del cliente y leerlo de la BD; rate-limit por IP/usuario (tabla o KV); clave en cabecera `x-goog-api-key`; `Deno.serve`; origen CORS restringido; devolver códigos HTTP reales. Verificar que `gemini-2.5-flash` siga vigente (no lo comprobé).

ID	Hallazgo	Ubicación	Gravedad	Consecuencia	Solución
A-04 [C]	**Ancla interna rota con HashRouter.** `href="#formulario-b2b"` cambia el hash y el router lo interpreta como la ruta `/formulario-b2b` → página 404.	`Business.jsx:122` (destino en `:244`)	ALTO	El principal CTA de la página B2B ("Solicitar catálogo y muestras") lleva a un 404.	Reemplazar por un `onClick` con `scrollIntoView` o usar BrowserRouter.
A-05 [C]	**Política de privacidad incompleta (Ley 1581/2012).** Menciona finalidad y derechos, pero no se evidencia razón social/NIT del responsable, canal y plazos de reclamo, ni transferencia internacional a Supabase y Google (Gemini).	`PrivacyPolicy.jsx`	ALTO	Riesgo regulatorio (SIC); además es un requisito de confianza.	Completarla con asesoría legal; añadir enlace y casilla en el chat y formularios; evaluar registro RNBD.
A-06 [C]	**Inconsistencia de marca.** El proyecto/ZIP/workflow/README dicen FRUTASO; todo el código, dominio (`fruites.co`), correo y redes dicen FRUITES. Los enlaces a redes apuntan a `instagram.com/` y `tiktok.com/` genéricos.	`siteConfig.js:13,35-39`, `metadata.json`, `deploy.yml`	ALTO	Confusión de marca, dominios/correos que quizá no existen, redes sin enlazar.	Definir el nombre definitivo, verificar disponibilidad de dominio, redes y marca ante la SIC, y centralizar el nombre en `siteConfig`.

Medios

ID	Hallazgo	Ubicación	Consecuencia	Solución
M-01 [C]	XSS almacenado potencial: el popup de Leaflet interpola `nombre`, `direccion`, `barrio`, `horario`, `categoria` sin escapar dentro de HTML.	`StoreMap.jsx` (~97-135)	Un registro malicioso en `stores` (o admin comprometido) ejecuta JS en todos los visitantes.	Construir el popup con nodos DOM/`textContent` o escapar; validar `lat/lng` numéricos.
M-02 [C]	Exportación CSV defectuosa: `encodeURI` no codifica `#` (el archivo se trunca en el primer `#` de un mensaje), `email` / `whatsapp` sin comillas, sin BOM (tildes rotas en Excel), sin defensa contra inyección de fórmulas (`=`, `+`, `-`, `@`).	`AdminDashboard.jsx:124-147`	CSV corrupto o peligroso al abrirlo.	Usar `Blob` + `URL.createObjectURL`, BOM UTF-8, comillar todo y prefijar `'` a celdas que empiecen por `= + - @`.
M-03 [C]	`INSERT ... WITH CHECK (TRUE)` en `contact_messages`: sin límites de longitud, formato ni tasa; solo honeypot en cliente.	`schema.sql:129-130`, `messagesService.js:47-49`	Spam y relleno de la base.	`CHECK` de longitud/formato; Turnstile/hCaptcha; función RPC con rate-limit.
M-04 [P]	Políticas admin basadas en `auth.email()` y en `OR` con `uid`; `FOR ALL` sin `WITH CHECK` explícito; `admin_users` por correo permite "reservar" un correo aún no registrado si el registro público está abierto sin confirmación.	`schema.sql:124-159`	Posible toma de privilegios en configuración laxa.	Basar todo en `auth.uid()`, desactivar registro público en Supabase Auth, activar confirmación de correo y MFA para admin, añadir `WITH CHECK`.

ID	Hallazgo	Ubicación	Consecuencia	Solución
M-05 [C]	Sin `ErrorBoundary` ni code-splitting: el público descarga admin, Leaflet y todas las páginas en un único chunk de 833 KB.	`App.jsx`, build	LCP/INP más lentos en móvil.	`React.lazy` por ruta (admin, mapa, recetas) + `Suspense` + `ErrorBoundary`.
M-06 [C]	8 JPG de 650–955 KB (≈6,5 MB), duplicados en `src/assets/images` y `public/images` (13 MB en el ZIP; los de `src/assets` no se importan en ningún lado); `<img>` sin `loading="lazy"` ni `width/height`.	`public/images`, `src/assets/images`, 8 `<img>`	LCP alto y CLS probable.	Convertir a WebP/AVIF (≈80–150 KB), `srcset`, dimensiones explícitas, `fetchpriority="high"` solo en el hero; borrar la copia no usada.
M-07 [C]	CSS de Leaflet cargado desde unpkg sin `integrity`, aunque `leaflet` ya es dependencia npm. Fuentes de Google sin autoalojar.	`index.html:14-19`	Dependencia de CDN de terceros, bloqueo de render.	`import 'leaflet/dist/leaflet.css'` y autoalojar fuentes (o usar `font-display: swap`, ya presente).
M-08 [C]	SEO técnico: un solo `<title>`/description; HashRouter hace que Google trate todo como una sola URL; sin `og:image`, canonical, favicon, sitemap, robots, JSON-LD; `/admin` sin `noindex`. `public/` solo contiene imágenes.	`index.html:6-13`, `public/`	Casi nulo posicionamiento orgánico y vistas previas pobres al compartir.	Migrar a BrowserRouter con fallback `404.html`, o a prerender/SSG; metadatos por ruta (`react-helmet-async`), sitemap, robots, OG image 1200×630, JSON-LD `Organization`/`Product`.
M-09 [C]	Dependencias y scripts sobrantes: `express`, `dotenv`, `@google/genai`, `tsx`, `esbuild`, `motion`, `@types/express` sin ningún import; `vite` y `plugin-react` en `dependencies`; script `clean` apunta a `server.js`; `name: react-example`; `__dirname` genera advertencia en Vite 8; restos de AI Studio (`DISABLE_HMR`, `metadata.json`).	`package.json`, `vite.config.ts`	Superficie de ataque y confusión innecesarias.	Limpiar `package.json` y `vite.config.ts`.
M-10 [C]	Claim nutricional que contradice la regla de marca: “concentra… la fibra, por lo que una porción pequeña es suficiente para saciar”. Además la web habla de “snacks saludables/preparaciones saludables”.	`recipes.js:132`, `Recipes.jsx:23`	Riesgo regulatorio de publicidad de alimentos (INVIMA/SIC) y coherencia de marca.	Retirar o validar con un profesional en normativa de alimentos antes de publicar.

Bajos

ID	Hallazgo	Ubicación	Solución
B-01 [C]	3 `<Link target="_blank">` del admin sin `rel` (mitigado: son rutas internas).	`AdminDashboard.jsx:306, 623, 760`	Añadir `rel="noopener noreferrer"`.
B-02 [C]	Enlace a `/admin/login` visible en el footer público.	`Footer.jsx:273`	Quitarlo o dejarlo oculto (la seguridad no depende de ello).
B-03 [C]	Sin `prefers-reduced-motion`; solo 1 regla `:focus-visible`; 10 `aria-label`. El chat sí tiene `role="dialog"`.	`styles/global.css`	Revisión de accesibilidad con axe/Lighthouse (no ejecutado).
B-04 [C]	El motor de chat demo usa coincidencias por palabra clave con orden fijo (p. ej. “tienda saludable” cae en la rama de salud).	`aiService.js:116-157`	Reordenar reglas o delegar al backend.
B-05 [C]	Texto literal `[POR DEFINIR]` aparece en datos y en respuestas del asistente.	`aiService.js:121`, `products.js`	Sustituir por “Próximamente” en la interfaz.
B-06 [P]	Sin cabeceras de seguridad (CSP, X-Content-Type-Options, Referrer-Policy). GitHub Pages no permite configurarlas.	Hosting	Considerar Cloudflare/Netlify/Vercel con `_headers`.
B-07 [P]	`supabase-js` guarda el token en `localStorage` (por defecto), lo que amplifica cualquier XSS futuro (ver M-01).	Cliente	Mantener XSS en cero y habilitar MFA.

4. Vulnerabilidades de seguridad (resumen por impacto)

1. **Autorización del admin en cliente (C-01).** *Impacto:* acceso a la interfaz sin credenciales. *Evidencia:* única comprobación `localStorage.getItem('fruites_admin_session')`. *Corrección:* sesión Supabase real + membresía en `admin_users` + MFA.
2. **Credencial de demostración pública (C-01).** *Impacto:* puerta de entrada conocida si el sitio sube sin variables de entorno. *Corrección:* compilar el modo demo solo en desarrollo y fallar el build de producción si faltan `VITE_SUPABASE_*`.
3. **Abuso de la Edge Function (A-03).** *Impacto:* consumo de cuota/costos y manipulación del asistente. *Corrección:* rate-limit servidor, prompt separado, catálogo desde BD.
4. **XSS almacenado en popup del mapa (M-01).** *Impacto:* ejecución de JS en el navegador de visitantes y robo de sesión de admin (token en `localStorage`). *Corrección:* escapar/usar DOM.
5. **RLS con puntos débiles (M-03, M-04).** *Impacto:* spam, posible escalada si el registro público está abierto. *Corrección:* basar en `uid`, cerrar registro, `CHECK` de datos.
6. **Inyección de fórmulas en CSV (M-02).** *Impacto:* ejecución de fórmulas al abrir el CSV exportado.
7. **Protocolos básicos OK:** no hay SQL propio (consultas parametrizadas por PostgREST → SQLi no aplica), CSRF no aplica (JWT en cabecera, sin cookies de sesión), no hay subida de archivos, HTTPS lo impone el hosting.

5. Funcionalidades: implementadas, simuladas y pendientes

Funcionalidad	Estado	Detalle
Navegación y rutas (13 páginas)	Implementada	Funciona por código; ancla B2B rota (A-04).
Catálogo + filtro por fruta	Implementada	No hay buscador de texto.
Detalle de producto	Parcial	Se rompe con datos de Supabase (C-04).
Recetas y detalle	Implementada (datos locales)	Imágenes dependen de la ruta base (A-01).
Mapa de tiendas (Leaflet/OSM)	Implementada, con datos ficticios	A-02, M-01.
WhatsApp / `wa.me`	Implementada	Número real configurado en `siteConfig.js:33-34`.

Funcionalidad	Estado	Detalle
Formulario de contacto	**Simulada**	No llega al negocio (C-02).
Formulario B2B	No verificado a fondo	Comparte servicio de mensajes; hereda C-02.
Chat/Asistente (modo demo)	Implementada (reglas)	Respuestas por palabras clave.
Chat con Gemini	Pendiente de integrar y probar	Requiere Supabase + secreto + endurecimiento (A-03).
Panel admin (mensajes, productos, tiendas, recetas)	Parcial/simulada	Guard débil (C-01), guardado local (C-03), CSV defectuoso (M-02).
Autenticación real	Pendiente de verificar	Existe `signInWithPassword`; no hay recuperación de contraseña ni MFA.
Redes sociales	Pendiente	URLs genéricas (A-06).
Carrito, checkout, pagos, inventario, pedidos	**No existen**	El sitio no es una tienda en línea: la venta se coordina por WhatsApp. No aplica la sección D de la auditoría; si se quiere e-commerce, debe diseñarse (pasarela, validación de precios en servidor, pedidos).
Notificaciones de nuevos mensajes	Pendiente	Ningún aviso por correo/WhatsApp al recibir un lead.
Respaldo y recuperación de BD	Pendiente	Sin política definida.

6. Plan de optimización

Prioridad 1 — Errores críticos y seguridad (≈ 3–5 días)

1. Arreglar instalación/despliegue (C-05): quitar `esbuild`, generar lockfile, `npm ci`.
2. Proteger el admin con sesión Supabase + `admin_users`, y retirar credencial/botón demo del build de producción (C-01).
3. Corregir el envío de contacto sin `.select()` y mostrar errores reales; eliminar el fallback fuera de demo (C-02, C-03).

4. Unificar el modelo de datos con capa `fromDb/toDb` y añadir `ErrorBoundary` (C-04).
5. Endurecer la Edge Function (A-03) y las políticas RLS (M-03, M-04); desactivar el registro público en Supabase Auth.
6. Retirar tiendas y mensajes ficticios del seed (A-02).

Prioridad 2 — Funcionamiento, estabilidad y UX (≈ 3–5 días)

1. Rutas de imágenes y limpieza de duplicados (A-01, M-06).
2. Corregir ancla B2B (A-04), exportación CSV (M-02) y popup del mapa (M-01).
3. Definir marca final y corregir dominio, correo y redes (A-06).
4. Completar política de privacidad y aviso en formularios/chat (A-05).
5. Sustituir `[POR DEFINIR]` por "Próximamente"; revisar claims (M-10, B-05).
6. Notificación de nuevos mensajes (Edge Function + correo/WhatsApp).

Prioridad 3 — Optimización, SEO y escalabilidad (continuo)

1. Code-splitting por ruta, WebP/AVIF, `lazy`, `width/height` (M-05, M-06, M-07).
2. Migrar de HashRouter a BrowserRouter o SSG/prerender; metadatos por ruta, sitemap, robots, OG, JSON-LD (M-08).
3. ESLint + Prettier, `checkJs` o migración a TypeScript, tests (Vitest + Playwright) y CI que ejecute build/tests.
4. Extraer estilos inline repetidos a clases/Tailwind; dividir `AdminDashboard.jsx` en módulos.
5. Hosting con cabeceras (CSP) y monitoreo; medir Lighthouse/Core Web Vitals reales (LCP < 2,5 s, INP < 200 ms, CLS < 0,1).
6. Limpiar dependencias (M-09) y activar Dependabot.

7. Checklist de lanzamiento

- [ ] `npm ci` y `npm run build` pasan en CI sin `--legacy-peer-deps`.
- [ ] Existe `package-lock.json` versionado; `package.json` limpio.
- [ ] Variables `VITE_SUPABASE_URL` y `VITE_SUPABASE_ANON_KEY` configuradas; el build **falla** si faltan.
- [ ] Modo demo, credencial demo y datos semilla eliminados de producción.
- [ ] Esquema SQL aplicado; RLS verificado con pruebas (anónimo, usuario normal, admin).
- [ ] Registro público de Supabase Auth desactivado; MFA para admin; correo de admin en `admin_users` por `uid`.
- [ ] Probado envío real del formulario (anónimo → fila en `contact_messages` → visible solo en admin).

- [ ] Edge Function con rate-limit, CORS restringido, secreto rotado/guardado y modelo vigente verificado.
- [ ] Todas las imágenes cargan en la URL final de producción (ruta base correcta).
- [ ] Ancla B2B, CSV y mapa corregidos; ErrorBoundary activo.
- [ ] Marca, dominio, correo y redes definitivos y funcionando; marca revisada ante la SIC.
- [ ] Política de privacidad y tratamiento de datos revisada por abogado; casillas de consentimiento en todos los formularios y en el chat.
- [ ] Claims y textos revisados por especialista en normativa de alimentos; sin `[POR DEFINIR]` visibles.
- [ ] SEO: metadatos por ruta, sitemap.xml, robots.txt, favicon, OG image, canonical, JSON-LD.
- [ ] Lighthouse móvil ≥ 90 en Performance/Accesibilidad/SEO/Best Practices.
- [ ] Prueba en iOS Safari, Android Chrome y escritorio (320, 375, 768, 1280 px).
- [ ] Cabeceras de seguridad configuradas (hosting con soporte) y HTTPS forzado.
- [ ] Plan de respaldo de Supabase y notificación de nuevos mensajes activos.

8. Conclusión profesional

Veredicto: NO APTO PARA PRODUCCIÓN. Apto únicamente para pruebas internas y demostraciones.

Justificación: el sitio compila y su interfaz es sólida, pero hoy (1) el despliegue automático falla, (2) el único formulario de captación de clientes no entrega los mensajes, (3) el panel admin no tiene un control de acceso real, (4) con Supabase conectado el catálogo y el detalle de producto se rompen por el desajuste de modelo, y (5) se muestran puntos de venta ficticios como si fueran reales.

No recomiendo reconstruir: la arquitectura base (React + Vite + Supabase) es adecuada y escalable. Basta corregir los componentes señalados, sobre todo la capa de servicios, el guard del admin, el esquema/mapeo de datos y la configuración de despliegue. Con la Prioridad 1 y 2 completas, podría salir como **“apto para lanzamiento con observaciones”**: landing de validación con contacto por WhatsApp y formulario funcional, sin e-commerce.

Limitaciones de esta auditoría: no se ejecutó el sitio en un navegador ni contra un Supabase/Gemini reales; los hallazgos marcados [P] deben confirmarse en un entorno de pruebas. No soy abogado: los puntos legales (A-02, A-05, A-06, M-10) son alertas para validar con un especialista.

Siguiente paso: quedo a la espera de tu autorización para comenzar las correcciones. Puedo trabajar por prioridades (P1 → P2 → P3) y, al terminar cada bloque, entregarte el resumen de archivos modificados, pruebas ejecutadas y pendientes.